

Gateway Framework for In-Vehicle Networks Based on CAN, FlexRay, and Ethernet

Jin Ho Kim, Suk-Hyun Seo, Nguyen-Tien Hai, Bo Mu Cheon, Young Seo Lee, and Jae Wook Jeon, *Member, IEEE*

Abstract—This paper proposes a gateway framework for in-vehicle networks (IVNs) based on the controller area network (CAN), FlexRay, and Ethernet. The proposed gateway framework is designed to be easy to reuse and verify to reduce development costs and time. The gateway framework can be configured, and its verification environment is automatically generated by a program with a dedicated graphical user interface (GUI). The gateway framework provides state-of-the-art functionalities that include parallel reprogramming, diagnostic routing, network management (NM), dynamic routing update, multiple routing configuration, and security. The proposed gateway framework was developed, and its performance was analyzed and evaluated.

Index Terms—Controller area network (CAN), Ethernet, FlexRay, framework, gateway.

I. INTRODUCTION

RECENTLY, the number of electronic control units (ECUs) in vehicles has been increasing to enable a variety of requirements and features, including those related to safety, fuel efficiency, convenience, and entertainment. Accordingly, the controller area network (CAN), FlexRay, and Ethernet have been proposed as in-vehicle network (IVN) protocols [1], [2]. CAN is a dominant protocol for IVNs, but it cannot provide real-time performance, which is essential in safety-critical applications, such as the x-by-wire system and advanced driver-assistance system (ADAS) [2]–[7]. To solve this problem, FlexRay was designed using a time-division multiple-access (TDMA) mechanism for safety-critical systems. Although FlexRay provides ten times the bandwidth of CAN, it cannot provide sufficient bandwidth for infotainment and multimedia [8]. Therefore, the multimedia-oriented system transport (MOST) was developed, but MOST has also become bandwidth constrained since 2013 due to an exponential increase in the bandwidth required for automotive systems [9]. Ethernet can provide the required bandwidth, and it has several advantages for use in IVNs, including low cost, high bandwidth, and mature

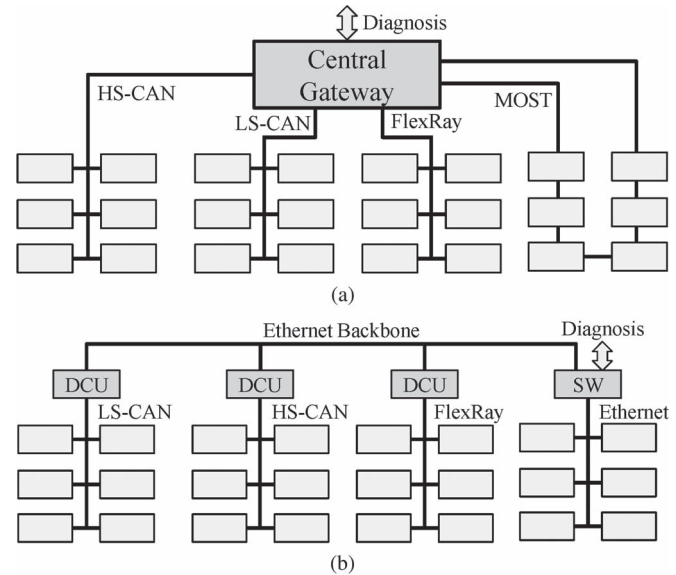


Fig. 1. Central gateway- and backbone-based IVN architecture. (a) Central gateway-based architecture. (b) Backbone-based architecture.

technology [10]. Therefore, Ethernet is expected to become one of the dominant protocols for IVNs in the future [9]–[12]. Ethernet can be used for diagnosis, multimedia, and infotainment, and it is expected to be used for ADASs and backbone networks in future automotive systems [9]–[12]. CAN, FlexRay, and Ethernet will be simultaneously used in the same vehicle after 2015 [9]. Therefore, a gateway is an essential component for seamless communication between CAN, FlexRay, and Ethernet.

The IVN architecture will change from a central gateway-based architecture to a backbone-based architecture [13], as shown in Fig. 1. In the central gateway-based architecture, a central gateway connects to the entire IVN [14] and provides seamless communication between heterogeneous network protocols. In the backbone-based architecture, a domain control unit (DCU) performs the role of the gateway between the backbone network and its own subnetwork. In this case, the number of gateways increases because a larger number of DCUs can be used, as shown in Fig. 1. Moreover, a gateway has to be implemented differently for each vehicle model or selected options for a specific vehicle model. Therefore, a gateway should be easy to develop using a variety of hardware and software platforms and should be easy to configure and verify.

A variety of gateways [15]–[21] have been proposed that use heterogeneous network protocols, including the local interconnect network (LIN), CAN, FlexRay, and Ethernet. A

Manuscript received July 22, 2014; revised October 26, 2014; accepted November 11, 2014. Date of publication November 20, 2014; date of current version October 13, 2015. This work was supported by the Priority Research Centers Program through the National Research Foundation of Korea funded by the Ministry of Education, Science, and Technology under Grant 2012-0005861. The review of this paper was coordinated by Prof. W. Song.

J. H. Kim, N.-T. Hai, B. M. Cheon, Y. S. Lee, and J. W. Jeon are with the School of Information and Communication Engineering, Sungkyunkwan University, Suwon 440-746, Korea (e-mail: besteng83@gmail.com; nguyentienhai@gmail.com; cheonbomu@naver.com; lys673@naver.com; jwjeon@yurim.skku.ac.kr).

S.-H. Seo is with the Electronic Architecture Development Team, Hyundai Motors, Hwaseong 445-706, Korea (e-mail: sh.seo@hyundai.com).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TVT.2014.2371470

CAN-CAN gateway with an optimized resource dimensioning method was proposed in [15]. Although it explains and analyzes a CAN-CAN gateway model, it did not address the problems of message conversion between heterogeneous protocols. A reliable gateway [16] resolves the message conversion problems between LIN, CAN, and FlexRay and focused on a reliable gateway mechanism using an OSEK/VDX network management (NM) [23], [24]. However, unlike our research, the reliable gateway did not consider state-of-the-art functionalities such as parallel reprogramming, dynamic routing update, and security, and it did not provide graphical user interface (GUI)-based configuration software and verification environments that would make it easy to reuse and verify for different vehicle models. Automotive open system architecture (AUTOSAR) [22] supports simple routing functionalities, including protocol data unit (PDU)-based routing and signal-based routing functionalities by PDU router and communication (COM) module, and these can be easily configured using AUTOSAR configuration tools. However, AUTOSAR does not provide the additional functionalities required for a gateway, such as parallel reprogramming and signal conversion rule. Additional functionalities should be implemented using a software component (SWC) or a complex device driver (CDD). The SWC implementation method can be used for a variety of platforms, but it decreases the performance and is not suitable for a gateway that requires strict real-time performance. In the case of using the CDD implementation method, a gateway can be implemented without a decrease in performance, but it may be difficult to reuse for a variety of hardware and software platforms. Moreover, AUTOSAR does not provide verification environment for the gateway, and it must be implemented. The mean and maximum routing delays for safety-relevant messages between CAN and FlexRay were measured using a data logging system in [17]. Accelerated routing mechanism using a field-programmable gate array (FPGA) to reduce the latency for a gateway was proposed in [18]. A fault-tolerant gateway using dual micro controller units (MCUs) was proposed in [19]. In this paper, a checker MCU checks the status of a main MCU, and if the checker MCU detects a problem of the main MCU, it runs minimum functions for safety, instead of the main MCU. A gateway between MOST, FlexRay, and Ethernet [20] has been proposed. This paper addresses a synchronous routing method that forwards the synchronous data (e.g., audio or video data) from MOST or Ethernet to FlexRay. A gateway for time-triggered control networks was proposed in [21]. Its prototype was developed using an FPGA and supported routing between FlexRay, Byteflight, time-triggered protocol/class C (TTP/C), and time-triggered CAN. Since the required functionalities and hardware/software platforms for a gateway are different depending on the vehicle model, previous gateways have had to be developed and verified for each individual vehicle model. Notwithstanding this variety of proposed gateways, the previous gateways were difficult to implement in a variety of vehicles because of the difficulty of reuse and verification; therefore, previous gateways required high cost and lengthy development times, and their reliability and safety were difficult to assure.

This paper proposes a gateway framework to solve the aforementioned problems. The proposed framework is designed to

be easy to reuse and verify for each individual vehicle model. As it is independent of the software and hardware platform, it can be easily implemented for various types of ECU that use a variety type of hardware/software platforms (e.g., ECU using a 32-bit MCU and AUTOSAR or ECU using a 16-bit MCU and OSEK/VDX). The proposed gateway framework can be configured using dedicated GUI-based software, which increases the ease of use, when developing and optimizing a gateway for specific vehicle models. The gateway framework can automatically generate a verification environment for the developed and optimized gateway for a specific vehicle model, and it is designed using an optimized AUTOSAR communication stack to improve performance and scalability. Therefore, the proposed framework can be used to easily develop and verify an in-vehicle gateway for various vehicle models. The proposed gateway framework provides the following nine features.

- The gateway framework supports frame-based routing, PDU-based routing, and signal-based routing between the CAN, FlexRay, and Ethernet.
- The gateway framework supports diagnostic routing between diagnostics over Internet protocol (DoIP) [25] and unified diagnostic services (UDSs) [26] and between UDSs that use different protocols (CAN and FlexRay).
- The gateway framework supports parallel reprogramming to reprogram several ECUs, which are connected to different network interfaces, in parallel to reduce the entire reprogramming time.
- The gateway framework supports both AUTOSAR NM and OSEK/VDX NM because both NMs are not compatible with each other [27]. The NM of each network interface in the gateway should be configured using configuration software. If the destination network is in a sleep state and a wake-up flag is set in the routing tables, the gateway framework should wake up the destination network and then transmit the message.
- The gateway framework can update only the routing tables and the configurations for the gateway without reprogramming the entire gateway firmware.
- The gateway framework can store multiple routing tables and configurations for the gateway to support a variety of types of vehicle models. Multiple routing tables and configurations should be selected before running.
- The gateway framework provides authentication to prevent unauthorized access and encryption/decryption to protect important data.
- The gateway framework should be easy to use to develop a variety of gateways. It can be configured and verified using GUI-based configuration software.
- The gateway framework should be safe. The gateway framework must be developed using static analysis techniques to improve the quality of the software, and it should be thoroughly verified. The gateway framework should support a callback function that can be programmed by the user for fail-safety when it detects serious problems.

The remainder of this paper is organized as follows. Section II explains the proposed gateway framework architecture, the operating mechanism, and the prototype gateway

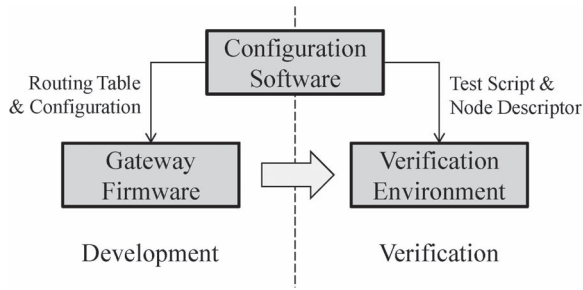


Fig. 2. Concept of the gateway framework.

framework that was developed. Section III evaluates the developed gateway framework, and Section IV concludes this paper.

II. GATEWAY FRAMEWORK

This section describes the proposed gateway framework and the prototype that was developed.

A. Concept of the Proposed Gateway Framework

The proposed gateway framework consists of configuration software, gateway firmware, and a verification environment. The configuration software can automatically generate routing tables and configurations for the gateway firmware, and test scripts and virtual node descriptors for the verification environment. The gateway firmware is reusable gateway software that supports state-of-the-art gateway functionalities and is independent of the hardware/software platform; thus, it is easily portable to various ECUs. Although the gateway firmware is reusable, the routing rules and configurations of a gateway differ according to the vehicle model and its requirements. Therefore, the routing rules and configurations have to be configured and automatically generated by using the configuration software. The verification environment supports automated verification of the developed gateway using test scripts and virtual node descriptors generated by the configuration software.

Fig. 2 shows the concept of the gateway framework. A gateway can be developed reusing the gateway firmware, and it is configured using routing tables and configurations generated by the configuration software. A variety of types of ECUs can be used for the development of the gateway because gateway firmware is independent of hardware/software platforms. After the development of a gateway, it can be verified using the verification environment that is configured using test scripts and virtual node descriptors. The configuration software generates the test scripts and virtual node descriptors according to the routing tables and configurations.

B. Architecture of the Gateway Firmware

Since the proposed gateway framework must be independent of the hardware and software platforms, it is designed based on a layered architecture that includes hardware, platform abstract, gateway framework, and the application layer. Fig. 3 shows the architecture of the proposed gateway firmware.

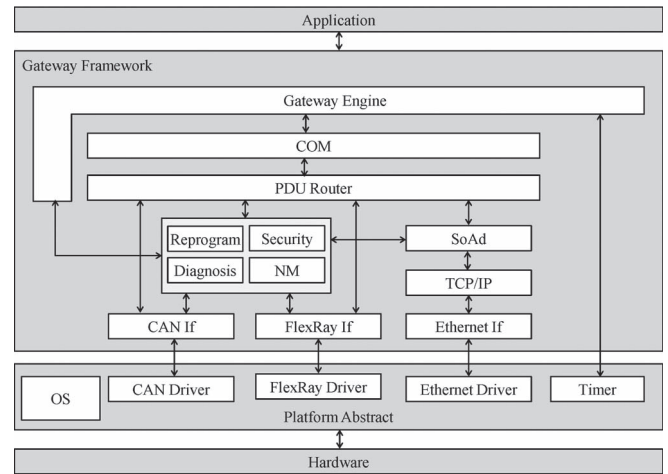


Fig. 3. Architecture of the proposed gateway framework.

A hardware layer contains the hardware components such as MCU and a communication controller. Since the requirements for a gateway are different for each vehicle model, the hardware design for the gateway can differ. The hardware layer interfaces only with the platform abstract layer and does not affect the gateway framework layer and the application layer.

To support the platform-independent requirement, a platform abstract layer is inserted between the hardware layer and the gateway framework layer. The platform abstract layer provides device drivers to control the hardware layer and a standard interface for the gateway framework layer. The platform abstract layer includes the CAN, FlexRay, Ethernet, and timer drivers and operating system (OS) modules, but it is possible to exclude or include these, depending on the hardware design or user requirements. The platform abstract layer must be developed taking the hardware layer into consideration because it is extremely dependent on hardware. To minimize the effort of implementation of the platform abstract layer, an AUTOSAR microcontroller abstraction layer (MCAL) [28] is used for the driver modules in the platform abstract layer since AUTOSAR MCAL provides all of the required device drivers with a *de facto* standard interface. Since a commercial off-the-shelf version of AUTOSAR MCAL is provided for almost all MCUs, which are mostly used in automotive systems, the effort required to develop the device driver module for the platform abstract layer can be minimized. The OS is optionally used, but it does not affect the gateway framework because it is used for the application layer (e.g., fail-safety function that developed by user), not for the gateway framework. The gateway framework can be delayed by the application when they are executed on the same processor, and it can cause an undesirable routing delay that can threaten the safety of the automotive system. To solve such a problem, a real-time OS with priority-based task management, such as the AUTOSAR OS or OSEK/VDX, should be used, and the priority of the gateway framework should be configured to be higher than that of an application program. If some application has to be given a higher priority than the gateway framework, such priority should be carefully considered, and the gateway should be thoroughly verified. This

problem can be solved by executing the gateway framework and the application on separate processors using dual-processor MCU. However, such a solution can increase costs.

The gateway framework layer consists of a gateway engine, a communication stack, and an add-on function module. The gateway engine configures and executes the required modules, according to routing tables and configurations. The gateway engine must be periodically executed by the timer module. A short execution period of the gateway engine can improve the performance of the gateway, but it requires more processing time and must be configured with consideration for the processing capability of the processor. The gateway engine periodically monitors the status of the other modules, and it calls callback functions when configured events (e.g., communication timeout or hardware error) occur.

The communication stack consists of a COM module, a PDU router, CAN interface, FlexRay interface, Ethernet interface, transmission control protocol (TCP)/IP, and socket adapter module. The detailed description for each module can be found in the AUTOSAR specification [22].

Add-on function modules include diagnosis, reprogramming, NM, and security modules. The diagnosis module includes DoIP, UDS, and its routing functions. The reprogramming module processes reprogramming request to different ECUs, which are connected to different network channels, at the same time to reduce reprogramming time. The NM module includes OSEK/VDX NM, AUTOSAR CAN NM, FlexRay NM, and UDP NM. The type of NM for each network is configured by configuration software. A security module provides the authentication algorithm and encryption/decryption algorithm.

The application layer includes a variety of user applications, which are developed by users using the information and functionality provided by the gateway framework.

C. Translation and Routing

The proposed gateway framework provides four types of routing methods, i.e., frame-based routing, PDU-based routing, signal-based routing, and diagnostic routing. A frame consists of a header, data, and a trailer. Data in the frame includes several PDUs, and the PDU also includes several signals. Frame-based routing routes the frame from the source network to the destination network. The PDU and signal in the source network are routed to the PDU and signal in the destination network by PDU-based routing and signal-based routing, respectively. Fig. 4 shows frame-based, PDU-based, and signal-based routing.

Frame-based routing between heterogeneous protocols requires additional translation processing because frames are protocol-dependent units. On the other hand, PDU-based routing and signal-based routing simply copy source data to the destination without an additional translation process since PDUs and signals are protocol-independent data units. Therefore, frame-based routing needs a translation process to convert the frame format (including header, data, and trailer) to that of the destination network protocol. It generates a frame for the destination network, considering the features of network protocols such as maximum length of the frame, transmission

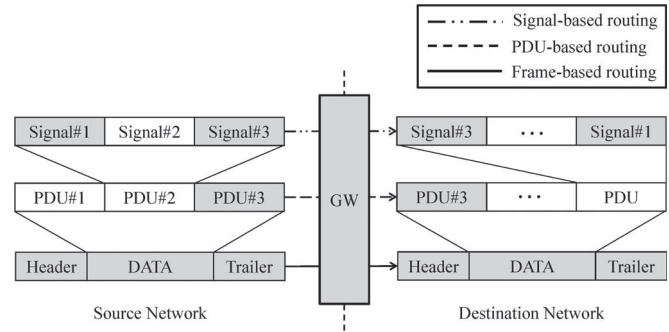


Fig. 4. Simple description of the routing methods.

TABLE I
FRAME ROUTING TABLE

Source Interface	ID	Destination Interface	ID
HS-CAN #1	0x4F1	FlexRay	0x51
HS-CAN #1	0x4F2	Ethernet	0x1000
FlexRay	0xD	HS-CAN #1	0x501
FlexRay	0xE	Ethernet	0x1100
Ethernet	0x1200	HS-CAN #1	0x501
Ethernet	0x1300	FlexRay	0xF

mechanism (e.g., TDMA or event driven), etc. Since the goal of the frame-based routing is to route as quickly as possible, it can be configured to route without searching a routing table by implementing it using switch statements in the interface module for every routing rule (i.e., it can reduce execution time by increasing code size). Occasionally, frame-based routing between different network protocols can be inefficient. Therefore, it is mostly used for routing between the same protocols (e.g., CAN-to-CAN gateway), and it is mostly used for diagnostic routing between CAN-based external diagnostic devices and CAN-based ECUs. This paper does not describe frame-based routing in detail because this has been already described in [16]. Table I shows the routing table for frame-based routing.

PDU is useful for network protocols, which support a longer frame length than CAN (8 bytes), such as FlexRay and Ethernet. In the case of Ethernet, for example, a frame can include multiple PDUs because the minimum frame length (64 bytes) of Ethernet is too long to store single data when considering that most of the control data for automotive systems are less than 8 bytes in length. Some automotive systems that use only CAN protocol do not use PDU (i.e., a frame consists of several signals, and a gateway only supports frame-based routing and signal-based routing). However, PDU and PDU-based routing are required when new protocols are used because a manufacturer has to resolve the message conversion problem between heterogeneous protocols by using a manufacturer-specific method if they do not use PDU and PDU-based routing. PDU and PDU-based routing are already specified in AUTOSAR, and the use of these is expected to increase in the near future.

PDU-based routing consists of direct and indirect routing. Direct PDU routing immediately transmits a received PDU to the destination network and is similar to frame-based routing, but it can route part of the frame (i.e., a PDU) to a destination

TABLE II
PDU ROUTING TABLE

Source PDU Name	Routing Type	Destination PDU Name
CAN_PDU_1	Direct	FR_PDU_3
CAN_PDU_2	Direct	Eth_PDU_3
FR_PDU_1	Direct	Eth_PDU_4
FR_PDU_2	Direct	CAN_PDU_3
Eth_PDU_1	Direct	FR_PDU_4
Eth_PDU_2	Direct	CAN_PDU_4

TABLE III
SIGNAL ROUTING TABLE

Source Signal Name	Destination Signal Name	Conversion Rule
CAN_SIG_1	FR_SIG_3	-
FR_SIG_1	Eth_SIG_3	
CAN_SIG_2	Eth_SIG_4	-
Eth_SIG_1	FR_SIG_4	$D=S/2+1$
Eth_SIG_2	CAN_SIG_3	-
FR_SIG_2	CAN_SIG_4	-

network without copying the entire frame. A received PDU that is configured to direct PDU routing is immediately forwarded to the destination interface module, such as the CAN or FlexRay interface, or to a socket adaptor by the PDU router module. On the other hand, indirect PDU routing just copies a received PDU to a destination PDU. The destination PDU can be transmitted when it meets a given transmission triggering condition according to the configured transmission mode: periodic, mixed, or direct [29].

Table II is the routing table used for PDU-based routing. The PDU routing table has to be sorted according to the type of protocol and message ID for the source network to reduce the searching time of the PDU routing table. The corresponding index of the PDU routing table for a received message and the number of related routing rules can be determined because scheduling and configuration of IVN are statically determined during development phase. Therefore, PDU-based routing does not need to search the entire routing table (i.e., search only pre-determined related routing rules). The configuration software automatically converts PDU name to PDU ID to optimize the routing table. The PDU ID is the unique identifier for the PDU that is used by the PDU router and the COM module. The configuration software also automatically configures the communication properties such as the transmission mode, cycle time, and length of the PDU according to the corresponding communication database (DB); thus, the PDU routing table does not include any information about communication properties.

Signal-based routing routes the signals from the source network to the destination network, according to the signal routing table shown in Table III. The configuration software automatically converts the signal name to the signal ID, which is the unique identifier of the signal used by the COM module. The configuration software also configures the communication properties for the signal according to the corresponding communication DB; thus, the signal routing table includes no information about communication properties. Unlike other routing methods, signal-based routing can change all communication properties, including the transmission period, transmission mode, and the value of the signal, because it is processed

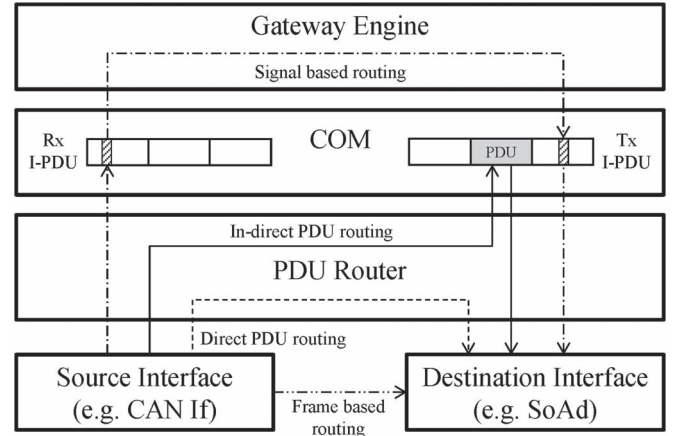


Fig. 5. Description of the data flow according to the routing method.

by the signal routing processing task that is executed in the upper layer of the COM module, as shown in Fig. 5, and is periodically called by the gateway engine. The signal routing processing task checks whether the source signal is updated or not by reading the signal routing table sequentially, and it routes the source signal to the destination signal if the source signal is updated. The signal is converted using a conversion rule if it is configured. Typically, the signal routing process task processes more than hundreds of routing rules and requires a long execution time. Therefore, the signal routing processing task should be a preemptive task to prevent the loss of received data caused by overflow of the receive interrupt queue.

Fig. 5 shows the data flow of the frame-based routing, PDU-based routing, and signal-based routing. Data for the frame-based routing is directly forwarded from the source interface to the destination interface. Data for direct PDU-based routing are forwarded to the destination interface via a PDU router. Data for indirect PDU-based routing are forwarded to transmit information PDU (I-PDU) [29] in a COM module via the PDU router, and the PDU is forwarded to the destination interface via the PDU router when the condition to transmit the PDU is met. Data for signal-based routing are forwarded to the receive I-PDU, and they are copied to the transmit I-PDU by the gateway engine. The copied signal is forwarded to the destination interface via the PDU router when the transmit condition of the PDU, which includes the copied signal, is met.

Diagnostic routing routes diagnostic messages between external diagnostic devices and ECUs, which are connected to the gateway using CAN or FlexRay. A gateway is connected to the external diagnostic device using CAN and Ethernet. DoIP is used as a diagnostic protocol for Ethernet, and UDS is used as a diagnostic protocol for CAN and FlexRay, because they are international standards for diagnostic protocols for automotive systems. To provide seamless communication between a DoIP-based external diagnostic device and UDS-based ECUs, a gateway needs a translation process between these different diagnostic protocols. Since diagnostic routing from CAN to CAN is a well-known technology and it can be simply implemented using frame-based routing, only diagnostic routing from Ethernet to CAN or FlexRay is described in this paper.

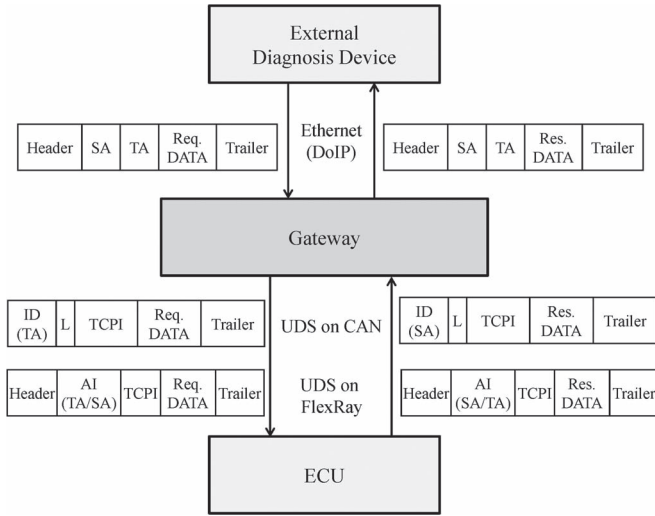


Fig. 6. Diagnostic routing between an external diagnosis device and the ECU.

TABLE IV
DIAGNOSTIC ROUTING TABLE

SA	TA	Sour. If	Socket ID	Dest. If
0x7E0	0x7E8	Ethernet	-	CAN
0x7E1	0x7E9	Ethernet	-	CAN
0x10	0x99	Ethernet	-	FlexRay

Fig. 6 shows the diagnostic routing between a DoIP-based external diagnostic device and a UDS-based ECU. A gateway checks the type field in the DoIP header when the gateway receives the DoIP message. If the value of the type field is that of a diagnostic message (0x8001), the gateway gets the value of the source address (SA) and the target address (TA) fields from the DoIP message, and it searches for the corresponding information from the diagnostic routing table described in Table IV. If the gateway successfully finds the corresponding routing information, it stores the socket ID, which can be used by a socket adapter module as an identifier for a particular socket connection and is dynamically assigned when each connection is established.

Then, the gateway creates a diagnostic message for the destination network protocol and transmits to the destination node (see ECU in Fig. 6). If the destination network protocol is CAN, the ID value of the created message is the value of the TA, and data are copied, without an additional translation process. The length of the CAN frame L is always 8 because every diagnosis message is transmitted using transport protocol (TP), irrespective of whether the length of the message is longer than the maximum length of the destination protocol or not. TP control information (TCPI) is determined by the CAN TP [30]. If the destination protocol is FlexRay, the FlexRay frame for diagnostic purpose would be already determined by scheduling. Typically, every node connected with FlexRay has one dynamic slot for the diagnostic purpose, but it can be different depending on scheduling. The diagnostic frame for FlexRay includes address information (AI), which includes TA and SA, and TCPI. TCPI is determined by the FlexRay transport protocol [31]. Detailed information about the communication sequence

between a DoIP-based external device and the UDS-based ECU can be found in [25], and frame mapping from DoIP to UDS for CAN and FlexRay can be found in [32] and [33].

When the destination node (see ECU in Fig. 6) receives the request message, it processes the request and sends a response message to the gateway. A gateway then sends the response DoIP message to the external diagnostic device after having received it from the ECU.

Table IV shows the routing table that stores the diagnostic routing configurations. It consists of SA, TA, socket ID, source interface, and destination interface.

D. Parallel Reprogramming and NM

A diagnostic system, which uses CAN as a diagnostic protocol, can update ECU software sequentially, one by one, due to limited bandwidth of CAN. As a result, it takes a lot of time to update every ECU that is contained in an automotive system. To solve this problem, Ethernet-based DoIP is used as a diagnostic protocol, and an external diagnosis device is connected to a gateway by using DoIP to transmit the reprogramming requests to a larger number of ECUs that are connected to the gateway via CAN or FlexRay. This is possible because Ethernet (100 Mb/s) provides much more bandwidth than CAN (1 Mb/s) and FlexRay (10 Mb/s). Parallel reprogramming can be used to simultaneously update the software on the ECUs connected to different networks on the gateway by using the increased bandwidth over Ethernet. This paper only describes the method for parallel reprogramming because reprogramming for an ECU (i.e., reprogramming ECU software sequentially) is a diagnostic service that is described in UDS, and it has been extensively used in automotive systems.

Since the inappropriate use of ECU software updates is a threat to the security and safety of automotive systems, authentication is required when reprogramming an ECU. After the authentication procedure is completed, an external diagnosis device can transmit the reprogramming requests simultaneously to ECUs connected to different networks on the gateway. When the diagnosis module included in the gateway receives a reprogramming request for the connected ECU, the gateway framework changes into a reprogramming mode and executes the reprogramming module. In the reprogramming mode, the gateway halts normal operation and attempts to route the reprogramming messages as quickly as possible. The gateway establishes a connection for each reprogramming request, and each reprogramming request is then stored in a buffer that is assigned to each connection. The gateway transmits the stored reprogramming request to the destination network by using TP. Parallel reprogramming is only different in that it can process a number of reprogramming requests simultaneously while the gateway converts the DoIP-based reprogramming request to a UDS-based reprogramming request that is similar to diagnostic routing. In the case where two 16-bit ECUs are connected to the gateway through different CAN channels that are configured to 500 Kb/s and where 617-Kb software is used to reprogram each ECU, sequential reprogramming would last for 120 s and parallel reprogramming would last 67 s, and in the latter case, the entire software update time is reduced by about 44%.

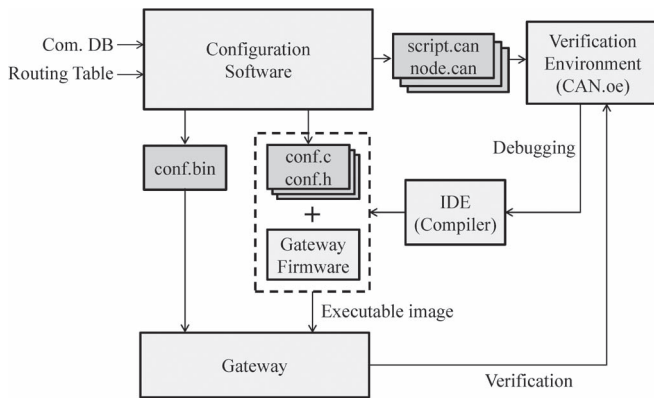


Fig. 7. Configuration and verification of the gateway framework.

NM is widely used to control the sleep and wake-up states of an ECU and a network. The gateway framework provides OSEK/VDX NM and AUTOSAR CAN NM for CAN, AUTOSAR FlexRay NM for FlexRay, and AUTOSAR UDP NM for Ethernet. Since OSEK/VDX NM has been widely used and it is not compatible with AUTOSAR CAN NM, both are provided. The type of NM and whether NM is used or not for each network are configured through the configuration software. A detailed description for each NM module is omitted in this paper because it can be found in the OSEK/VDX specification [23] and AUTOSAR specification [22].

These NM modules provide information on the states of the network and nodes connected to the network, whether these are in a sleep or a wake-up state. Every routing rule has a wake-up routing (WU) flag, and it can be configured by the user. If a WU flag is true, the gateway transmits the source message to the destination after the wake-up of the destination network when the destination network is in a sleep state. In the case where the WU flag is false, the gateway ignores the source message if the destination network is in a sleep state.

E. Configuration and Verification

A gateway is an important ECU because it affects the safety of an automotive system. Therefore, factors that can cause an error should be minimized during the development process, and the gateway should be thoroughly verified. The proposed gateway framework provides the configuration software and verification environment to solve these issues.

The configuration software is a GUI-based program for configuration of the gateway firmware and generates the configuration files that include routing tables and configurations for the gateway firmware and test scripts and virtual node descriptors for the verification environment. Test scripts and virtual node descriptors must be generated by the configuration software for accurate verification of the gateway because the gateway operates according to the routing tables and configurations generated by the configuration software.

Fig. 7 describes the configuration and verification method of a gateway using the configuration software. The configuration software automatically configures a gateway framework by reading communication DB, such as CAN DB or field bus

exchange format (FIBEX) [34], and a routing configuration file that can be written using Excel or extensible markup language (XML) file format. GUI-based configuration software is used to manually configure the interface between the gateway framework and platform-independent layer. For example, B-CAN that is a CAN network configured by a communication DB has to be assigned to a real communication channel, and the NM for each network has to be configured. After configuration, the configuration software can generate configuration files, test scripts, and virtual node descriptors. The configuration software generates the configuration as a binary file or as source files. Configuration source files should be compiled with the gateway firmware to build an executable image that can be executed on the gateway. This form of configuration file can be used for debugging to fix an error in the development process of the gateway. A configuration binary file is commonly used to support a dynamic routing update function after the development of the gateway has been completed. A developed gateway can be verified using the test scripts and virtual node descriptors generated by the configuration software. If defects of the developed gateway are found during the verification process, debugging of the gateway can be possible using the configuration source files.

A binary configuration file can be written to a specific memory area in the gateway without modification of the memory area where the gateway firmware is stored (i.e., the memory areas for the gateway firmware and the configuration are separated). The gateway firmware reads the configuration from the specific memory area before initializing the gateway. This function, called dynamic routing update, allows the user of a gateway framework, such as an original equipment manufacturer (OEM) or Tier 1, to develop the gateway without deep understanding of the gateway firmware. Moreover, a developed gateway, which supports a dynamic routing update function, can be used for various vehicle models by changing only the configurations of the gateway framework without developing a gateway for each model.

The configuration software can generate the configuration files that include multiple routing tables. A gateway can selectively use multiple routing tables. This function, called multiple routing tables, is required for the gateway to support various vehicle models or a variety of options for a single-vehicle model. To minimize the memory usage, multiple routing tables have to be compressed. For example, a simple compression algorithm that assigns an ID to every routing rule and replaces repetitive rules by ID can be used. Compressed routing tables do not degrade the performance of the gateway because they are decompressed before the operation of the gateway.

The verification environment uses test scripts and virtual node descriptors to verify the gateway. CAN.oe, which is a network monitoring and simulation tool developed by Vector, is used as the base environment to implement a verification environment for convenience in implementation. The verification environment must provide the same communication environment as that of a real vehicle, where a gateway is connected to a number of ECUs. For this purpose, the configuration software generates virtual node descriptors. The virtual node descriptor is an executable script that performs the role of a

TABLE V
TEST CASE OF THE VERIFICATION ENVIRONMENT

Test Case	Description
Routing rules	Check that the gateway routes frame, PDU, and signal conforming to configured routing rules by checking input and output of the gateway
Gateway latency	Check that the gateway routes message, PDU, and signal within the specified latency
Data conversion	Check that the gateway routes the signal with changing the value according to the conversion rule
Com. configuration	Check that the gateway transmits frame, PDU, and signal conform to the given configured transmission mode (direct, mixed, periodic)
Time-out processing	Check that the gateway transmits the pre-determined time-out value or calls callback function according to configuration when a time-out occur
Diagnostic routing	Check that the gateway routes diagnostic message conforming to diagnostic routing rules

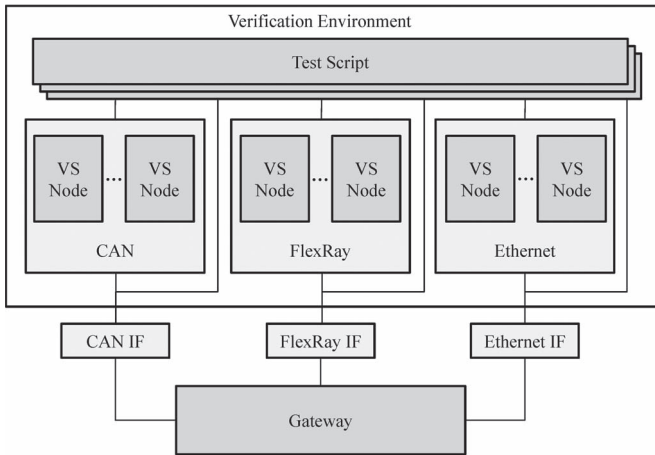


Fig. 8. Verification environment for the gateway framework.

virtual ECU for the verification environment. Although an ECU uses complicated algorithms, a virtual ECU imitates only the network-related features. A virtual ECU can send and receive messages, much like a real ECU, but the values of the messages are not the same. The configuration software can generate the virtual node descriptors using only the communications DB (CAN DB or FIBEX), and the values of the data are randomly assigned during runtime of the verification environment.

The configuration software generates test scripts for each of the test cases described in Table V. A gateway ECU, which is mounted on the vehicle after completion of the development, has a variety of functions in addition to routing-related functions. These functions are not provided by the proposed gateway framework because they may be different for each gateway manufacturer or OEM. Therefore, the verification environment of the gateway framework supports only the verification of routing-related functions.

Fig. 8 describes the verification method used by the verification environment. It creates a virtual network for each individual network connected to the gateway. For example, five virtual CAN networks are created if the gateway is connected to five CAN networks in the vehicle. The virtual network includes a number of virtual nodes, and both are configured using the

communication DB by the verification environment. CAN.oe provides a basic communication environment to communicate according to the communication DB. However, the virtual nodes do not include application algorithms, which determine the transmission timing and value of the data. To solve this, a virtual node descriptor is assigned for every virtual node, and each virtual node descriptor provides an algorithm, which determines the timing and value of the message. To support various test cases, virtual node descriptors support various modes for each test case, and it can be selected by the test scripts.

The test scripts change the mode of the virtual nodes before verification to make virtual nodes suitable for each test case, and the scripts perform verification using the messages transmitted on the communication channel between the virtual node and a gateway. The interface modules provide the interface between the verification environment and each CAN, FlexRay, or Ethernet communication channel, and these provide a precise timestamp using special hardware when a message is received or transmitted.

The test script for verification of the gateway latency, for example, changes the mode of the virtual nodes to verification latency mode. Then, virtual nodes transmit the message using a corresponding interface module according to the virtual node descriptor. The verification environment notifies the fact that the message is transmitted and its transmission time. The test script stores the transmission time and waits to receive a converted message from the gateway. The message transmitted from the virtual node is then converted and transmitted to the destination network by the gateway according to the configured routing rules. The verification environment receives the converted message via an interface module and notifies the test script of the fact that a message has been received and its reception time. The test script can calculate the gateway latency time by subtracting the transmitted time from the received time. The test script checks that the calculated gateway latency time is less than a predetermined gateway latency time. If the calculated gateway latency time is greater than the predetermined gateway latency time, a failure of the verification is notified and the information is stored for error reporting. The verification process is performed repeatedly and simultaneously for every configured routing rule.

After all verification has been completed, the verification environment reports the success or failure of each test case, and it generates report files for error tracing.

F. Prototype Implementation

The proposed gateway framework can be easily developed for a variety of platforms, but in this paper, it is implemented using an MPC5668G evaluation board (MPC5668EVB) [35]. The MPC5668EVB provides six channels for CAN, one channel for FlexRay, and one channel for Ethernet. Fig. 9 shows the MPC5668EVB used to implement a prototype gateway. AUTOSAR MCAL 3.0 and AUTOSAR OS are used for the hardware abstract layer to interface between the gateway framework and hardware. Tresos Studio 2010.a, which was developed by Elektrobit, is used as the configuration tool for AUTOSAR MCAL and AUTOSAR OS. Since AUTOSAR

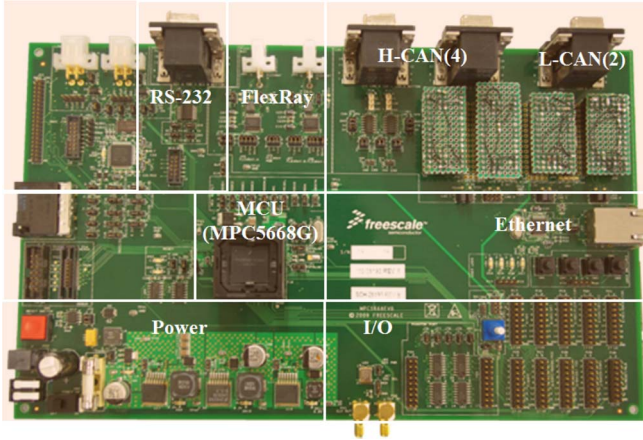


Fig. 9. Embedded system used for prototype implementation.

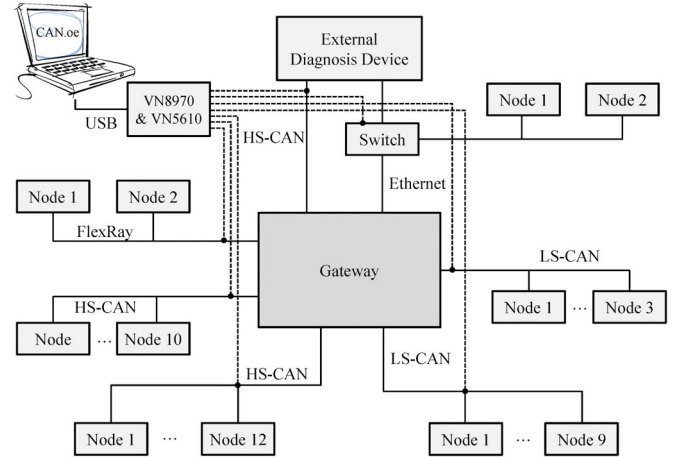


Fig. 11. Evaluation environment for the prototype gateway.

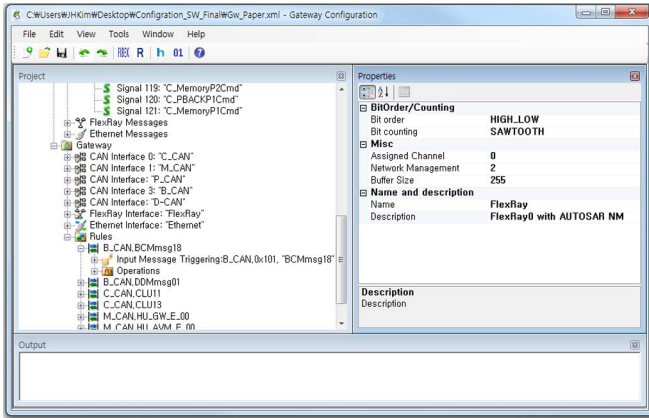


Fig. 10. Developed configuration software.

MCAL 3.0 does not provide Ethernet, only the essential functions of the Ethernet driver were developed.

The proposed gateway framework can be easily ported to a variety of platforms without extra work since it is designed to be compatible with AUTOSAR MCAL. However, the gateway engine, as described in Fig. 3, has to be called on periodically using a timer. In our implementation, the timer is implemented using the alarm service provided by AUTOSAR OS and is called every 1 ms.

Fig. 10 shows the implemented configuration software for the proposed gateway framework. The configuration software is developed using .NET, and it can automatically generate the configuration files, test scripts, and virtual node descriptors for the gateway framework from the FIBEX and routing configuration file. The routing configuration file is written using Excel (developed by Microsoft), and it includes the routing tables described in Tables I–IV, and additional information, including the predetermined maximum latency time, wakeup flag, timeout time, and timeout value, is added for each routing rule.

The configured routing tables for the prototype gateway are similar to that of a commercial vehicle, as shown in Fig. 11. The prototype gateway is connected to five CAN channels, a FlexRay channel, and an Ethernet channel. CAN and FlexRay are connected to the CAN.oe using a VN8970, and Ethernet is connected to the CAN.oe using a VN5610. VN8970 and

TABLE VI
CONFIGURATION OF THE USED PROTOCOL

LS-CAN		HS-CAN	
PROPSEG	2	PROPSEG	2
PSeg1, PSeg2	1, 3	PSeg1, PSeg2	1, 3
Baud rate	100 Kbps	Baud rate	500 Kbps
FlexRay			
Cycle length	5 ms	Macrotick(MT)	1 μ s
No. of static slot	75	No. of mini slot	293
Static slot length	43 MT	Dynamic slot length	6 MT
Static Payload Length	20 bytes	Baud rate	10 Mbps
Ethernet			
Interface mode	MII	Full/Half duplex	Full duplex
TCP/IP Stack	LwIP 1.4.0	Baud rate	100 Mbps

VN5610 are universal serial bus (USB)-to-IVN interface modules developed by Vector. CAN, FlexRay, and Ethernet are configured using parameters, as shown in Table VI. The prototype gateway was implemented using the embedded system shown in Fig. 9. The 39 nodes are connected to the gateway, and these were implemented as virtual nodes on the CAN.oe-based verification environment. A PC-based OEM-specific diagnostic program was used as an external diagnostic device.

III. EVALUATION AND ANALYSIS

The proposed gateway framework has to be safe, and it has to be reusable. Here, the reusability and safety of the gateway framework are evaluated, and its performance is analyzed and measured.

A. Reusability

To improve the reusability of the gateway framework, it was designed using a platform-independent architecture, and it can be easily ported to a variety of platforms by using AUTOSAR MCAL as the platform abstract layer. Moreover, the routing tables and configurations, which differ for each individual vehicle model, are configured and generated by using GUI-based configuration software. This can generate configuration files that include routing tables and configurations for the gateway firmware by reading the communication DB and the routing

configurations. Therefore, the gateway can be configured without understanding the gateway firmware. Moreover, using GUI-based configuration software can reduce human error for configuration of the routing rules and configurations for the gateway. Since a recent luxury car may use more than 70 ECUs and communicate more than 2500 signals [36], the routing rules for a gateway are too complex to configure manually.

The gateway framework improves reusability by supporting an automated verification process by using the verification environment that uses test scripts and virtual node descriptors automatically generated by the configuration software. The reliability of the gateway is very important, and the gateway must be thoroughly verified to assure reliability. The proposed gateway framework provides a verification environment; hence, the development time for the verification environment of a gateway can be reduced, and a more accurate verification is possible by reusing the well-designed verification environment supported by the gateway framework.

Thus, in comparison with previous gateways [16]–[21], the gateway framework provides improved reusability by providing well-designed reusable gateway firmware, configuration software, and verification environment.

B. Safety

The gateway framework is a safety-critical software component, and software quality assurance is important for the safety of passengers. For this purpose, static analysis techniques [37] are used to develop the gateway framework. These techniques prove that the software fulfills its specification and verify that the software does not have violations of recommended programming practices, which may cause incorrect behavior of the system [38].

Two types of static analysis methods are used. In the first, we check the motor industry software research association (MISRA) C coding rules. MISRA C is a safer language subset that prevents dangerous behavior of embedded control systems in automotive applications [39], [40], which is widely used in the automotive industry. Our implementation conforms to MISRA C:2004 [42], and PC-LINT 9.0 (developed by Gimpel Software) and LDRA 9.2.0 (developed by LDRA Software Technology) are used to verify MISRA C:2004 conformance. Every violation was eliminated with the exception of incorrectly detected violations, but these were carefully reviewed.

The other method is checking for potential defects using static analysis. CodeSonar (developed by Gramma Technology) is used for this method. CodeSonar can detect potential defects such as race condition, null pointer de-references, process starvation, and buffer overflow by examining the dataflow and symbolic execution of the software. Detailed descriptions of CodeSonar are provided in [42]. Every critical and major defect detected by CodeSonar was eliminated, and every minor defect is reviewed by the developer.

The proposed gateway framework must be verified with integrating the gateway hardware. Although the proposed gateway framework provides a verification environment for the developed gateway, it is also necessary to verify the verification environment itself, which is provided by the gateway

framework. Therefore, the prototype gateway is verified using a verification environment developed by another organization that manufactures the commercial gateway ECU. The verification environment generates the verification environment from the routing DB (RDB) and the communication DB and then verifies whether the gateway works as configured in the RDB and the communication DB in a manner similar to that of the verification environment provided by the proposed gateway framework. The evaluation environment is similar to that in Fig. 11; however, only CAN is used for this verification.

Moreover, the proposed gateway framework supports the reliability mechanism described in [16]. The gateway framework does not transmit any messages to failed (or fault) nodes that cannot receive the message. The gateway framework provides callback functions for the fail-safety mechanism when the gateway framework detects serious problems such as detecting a fail node that is connected to the gateway via a network or detecting hardware problems for the gateway.

C. Performance

To evaluate and analyze the performance of the proposed gateway framework, the worst case of end-to-end processing time is theoretically analyzed and measured. The end-to-end processing time is the time duration from the instant the source message is transmitted at the source network to the instant the converted message is transmitted to the destination network.

1) *Theoretical Analysis*: The end-to-end processing time is flexible with respect to the communication protocols, communication speed, transmission mode (e.g., direct, mixed, and periodic), and type of routing method. Therefore, the worst case of end-to-end processing time of the gateway is analyzed using the following notations:

$W(x)$	worst-case execution (or processing) time for x ;
$T_e(x)$	execution time for x ;
$T_{\text{wait}}(x)$	wait time for calling x .

The worst-case end-to-end processing time T_{e2e} for the gateway is determined by (1) as follows:

$$W(T_{e2e}) = W(T_{\text{rx}}) + W(T_{\text{proc}}) + W(T_{\text{tx}}) \quad (1)$$

where T_{rx} is the time duration for receiving the source message, T_{proc} is the time duration for processing the received message, and T_{tx} is the time duration for transmitting the processed message to the destination network.

T_{rx} includes the time spent by the communication controller to receive the source message and the time spent by the interrupt service routine (ISR) to copy the received message to the corresponding buffer. The worst case of T_{rx} is denoted by

$$W(T_{\text{rx}}) = T_{\text{cc_rx}} + W(T_{\text{wait}}(\text{ISR}_{\text{rx}})) + T_e(\text{ISR}_{\text{rx}}) \quad (2)$$

where $T_{\text{cc_rx}}$ is the required time to receive the message from the source network by the communication controller, and ISR_{rx} is a receive ISR for the received message.

Since the transmission of a message by the source node and the reception of a message by the gateway are simultaneously performed, $T_{\text{cc_rx}}$ can be ignored (i.e., the value of $T_{\text{cc_rx}}$ is

zero). $T_e(\text{ISR})$ can differ depending on the communication protocol or the length of the received message, but it can be assumed to be constant because it is a nonpreemptive task, and the difference is generally small. A requested receive ISR by the communication controller is called after terminating all nonpreemptive tasks, including executing nonpreemptive tasks and pending ISRs that are requested before the receive interrupt. Therefore, the worst case execution time of $T_{\text{wait}}(\text{ISR})$ is determined as follows:

$$W(T_{\text{wait}}(\text{ISR})) = T_e(\text{Task}_{\text{np}}) + \sum_{i=1}^n T_e(\text{ISR}_i) \quad (3)$$

where Task_{np} is the nonpreemptive task that is already running when the receive interrupt is requested, and n is the number of the pending interrupts that were requested before the receive interrupt.

The routing processing task $\text{Task}_{\text{proc}}$ for the signal-based routing is periodically called, and it sequentially copies the source signal to the destination signal when the source signal is updated. The worst case of T_{proc} for signal-based routing is when the routing information is located in the last index of the routing table, and every signal is updated. The worst case of T_{proc} for signal-based routing is determined by

$$W(T_{\text{Proc}}) = \sum_{i=1}^k (\theta + T_c) + W(T_{\text{wait}}(\text{Task}_{\text{proc}})) \quad (4)$$

where k is the last index of the routing table, θ is the time duration for comparing the ID of the received message to the ID of the routing table, and T_c is the time duration for converting the source message to the destination message. T_c can differ according to communication protocols and the length of message, but it can be also assumed to be constant because the difference is generally small.

In the case of PDU-based routing, k is the largest number of the related routing rules to process the received message. In the case of frame-based routing, k is 1. $T_{\text{wait}}(\text{Task}_{\text{proc}})$ for frame-based routing and PDU-based routing can be ignored (i.e., it is zero) since the gateway processing task is directly called by receive ISR.

The worst case of $T_{\text{wait}}(\text{Task}_{\text{proc}})$ for signal-based routing is when it is executed in the next period, and the maximum number of preemptive tasks is executed while $\text{Task}_{\text{proc}}$ is executing. $\text{Task}_{\text{proc}}$ can be preempted by other tasks that have higher priority because the execution time for $\text{Task}_{\text{proc}}$ is too long to be a nonpreemptive task. Typically, more than hundreds of routing rules are configured for signal-based routing. If $\text{Task}_{\text{proc}}$ is configured as a nonpreemptive task, loss of the data may occur by overflow of the receive interrupt queue due to its execution time being too long. Therefore, signal-based routing cannot provide real-time routing. If some signals must be routed with a strict time deadline, these signals must be routed using direct PDU-based routing or frame-based routing. The worst case of $T_{\text{wait}}(\text{Task}_{\text{proc}})$ is determined by

$$W(T_{\text{wait}}(\text{Task}_{\text{proc}})) = T_{\text{period}} + \sum_{i=1}^m T_e(\text{Task}_p(i)) \quad (5)$$

where T_{period} is the time period calling the signal routing process task, and m is the maximum number of the preemptive tasks that can be executed while $\text{Task}_{\text{proc}}$ is executing.

The worst case T_{tx} is determined by

$$W(T_{\text{tx}}) = T_{\text{cc_tx}} + W(T_{\text{wait}}(\text{Task}_{\text{tx}})) + T_e(\text{Task}_{\text{tx}}) + W(T_{\text{protocol}}) \quad (6)$$

where $T_{\text{cc_tx}}$ is the time duration for transmission of the processed message to the destination network by the communication controller, Task_{tx} is the task that is responsible for copying from the destination buffer to the transmit buffer in the communication controller, and T_{protocol} is the time delay caused by the CAN, FlexRay, and Ethernet protocol characteristics.

$T_{\text{cc_tx}}$ can be calculated by taking the total number of bits of data, including overhead, such as header and stuffing bit, and dividing by the communication speed (e.g., 100 Mb/s for Ethernet). $T_{\text{cc_tx}}$ is determined by the communication speed and the number of bits of data to be transmitted, regardless of the performance of the gateway.

Task_{tx} is called differently depending on the type of routing method. In the case of frame-based routing and direct PDU-based routing, it is called directly in the gateway processing task ($\text{Task}_{\text{proc}}$). In this case, $T_{\text{wait}}(\text{Task}_{\text{tx}})$ is zero. In the case of the indirect PDU-based routing and signal-based routing, Task_{tx} is called by the COM module. In this case, the COM module calls Task_{tx} when the configured transmission condition is met by periodically checking the configured transmission condition of the PDU or the signals. The worst case of the waiting time for calling Task_{tx} is the time period of the process task for COM module, if PDUs or signals are configured in direct transmission mode, or is the transmission period of the PDU or signal, if these are configured in periodic transmission mode.

T_{protocol} can vary depending on the destination network protocols. In the case of a CAN, T_{protocol} is the time delay caused by the ID-based arbitration. Typically, it is difficult to determine T_{protocol} for CAN due to its nondeterministic feature. Detailed information for the analysis of T_{protocol} for CAN is described in [43]. The T_{protocol} for FlexRay is caused by the cyclic communication method using TDMA, and the worst case of the T_{protocol} for FlexRay is the communication cycle of FlexRay. In this case, data are transmitted during the next communication cycle. T_{protocol} can be affected by communication scheduling of the FlexRay, and detailed descriptions for the scheduling of FlexRay are described in [44]–[47] and performance analysis of FlexRay are described in [48] and [49]. Since Ethernet does not use ID-based arbitration or a TDMA mechanism, these time delays do not occur when using Ethernet. However, Ethernet can be delayed by a maximum of 122 μs , which is the time required to transmit the maximum length (1514 bytes) of the frame for 100 Mb/s Ethernet, since once transmission has begun, transmission of the Ethernet frame cannot be preempted by a frame with higher priority. This means that a frame with higher priority can be delayed if the lower priority frame is already being transmitted. This problem can also occur for CAN, and it can be delayed by a maximum of 270 μs , which is the time required to transmit the maximum length (8 bytes) of the frame for 500 Kb/s CAN. In this analysis, priority arbitration in the

transmission buffer of the communication controller is ignored to simplify the analysis.

According to this analysis, the gateway end-to-end processing time can be delayed by nonpreemptive tasks or interrupts, and therefore, their use should be minimized. Some interrupts, such as received interrupts for CAN, FlexRay, and Ethernet, are essential to enable efficient communication. In this case, the ISR must be optimized to reduce the execution time. Moreover, it is recommended not to use nonpreemptive tasks and interrupts for user-specific applications that are developed by the user on the application layer. Nonpreemptive tasks or interrupts must only be used after careful consideration.

2) *Measured Performance and Analysis:* The prototype gateway described in Section III is used to measure the performance of the proposed framework. The verification environment described in Fig. 11 is used to measure the performance of the gateway, but only one high-speed CAN, FlexRay, and Ethernet are used for this measurement.

The end-to-end processing time of the prototype gateway for signal-based routing, PDU-based routing, frame-based routing, and diagnostic routing is measured using a CAN.oe-based verification environment, as shown in Fig. 11. The measured end-to-end processing time T_{m_e2e} is defined by

$$T_{m_e2e} = S_{rd} - S_{rs} \quad (7)$$

where S_{rd} is the timestamp when VN8970 or VN5610 receives the converted message on the destination network, and S_{rs} is the timestamp when VN8970 or VN5610 receives the source message on the source network. S_{rd} and S_{rs} are precisely measured within 1 μ s error by VN8970 and VN5610.

The CAN.oe-based verification environment has about ± 400 μ s jitter to transmit an Ethernet message. This means that the actual transmission period can change between 9.6 and 10.4 ms, although an Ethernet message is transmitted every 10 ms in CAN.oe. This time jitter makes it difficult to precisely measure the end-to-end processing time. To solve this problem, additional embedded systems are used to transmit the source message to the gateway. The measured time jitter of the transmission period of the message transmitted by the additional embedded system is less than ± 1 μ s.

To measure the end-to-end processing time of the diagnostic routing, the routing rules described in Table IV are used. The diagnostic routing is performed twice for a request from an external diagnostic device to ECU. First, one is performed to route a DoIP request from the external diagnostic device to a destination node that is connected to either the CAN or FlexRay network. The other is performed to route the response diagnostic message from the destination node to the external diagnostic device. Fig. 12 shows the measured end-to-end processing time for the diagnostic routing, which is less than 1.18 ms for that between Ethernet and CAN and less than 10 ms for that between Ethernet and FlexRay. The latter is about twice of the communication cycle of FlexRay because one communication cycle is used to transmit the request message to the destination node, and the other communication cycle is used to transmit the response message from ECU to the external diagnostic device. The measured end-to-end processing time of diagnostic routing for FlexRay increases, as shown in Fig. 12,

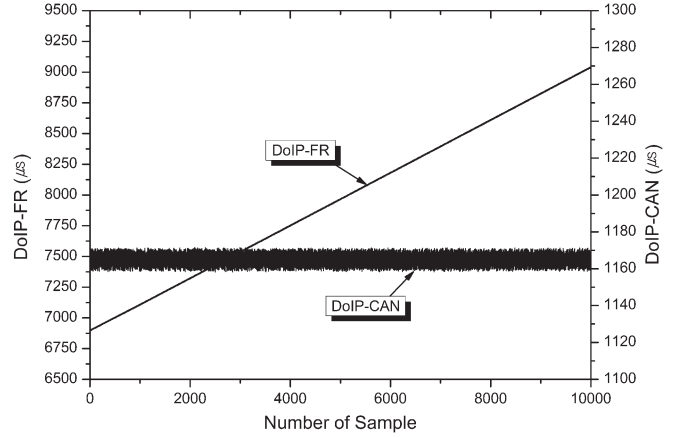


Fig. 12. Measured end-to-end processing time for diagnostic routing.

because the clocks of the external diagnostic device and the gateway are not synchronized, and a clock drift problem [50] occurs. In this measurement, the clock of the external diagnostic device is faster than the clock of the gateway. Therefore, the gateway receives the DoIP request message earlier, although the external diagnostic device sends a DoIP request message every 10 ms.

To measure the end-to-end processing time of the frame-based routing, PDU-based routing, and signal-based routing for CAN, FlexRay, and Ethernet, the routing rules described in Tables I–III are used. Every PDU used for PDU-based routing is configured to direct transmission mode, and every signal for signal-based routing is configured to periodic transmission mode with 10 ms transmission cycle. The frame lengths for CAN, FlexRay, and Ethernet are configured to 8, 20, and 64 bytes, respectively. Although the frame length for FlexRay and Ethernet is longer than 8 bytes, only the first 8 byte data are used for frame-based routing to prevent multiple (fragmented)-frame transmission. The length of every PDU and signal, which are used for this measurement, are configured to 8 bytes. Two additional embedded systems (one for CAN and Ethernet, other for FlexRay) are used to transmit the source message to the gateway every 10 ms, and the CAN.oe-based verification environment measures the end-to-end processing time.

Fig. 13 shows the measurements and the analysis results for the end-to-end processing time, according to the routing method. $T_e(\text{Task}_{np})$, θ , T_c , $T_e(\text{Task}_{tx})$, $T_e(\text{ISR}_{rx})$, T_{cc_tx} , and T_{protocol} are measured to analyze the worst case end-to-end processing time of the developed gateway. The measured worst cases for $T_e(\text{Task}_{np})$, θ , T_c , and T_{period} are 5, 1, 4 μ s, and 1000 μ s, respectively. $T_e(\text{Task}_{tx})$ can slightly vary according to the destination network and the data length. However, we assume that this value is 20 μ s to simplify the analysis. $T_e(\text{ISR}_{rx})$ can also vary according to the routing method and source network. The measured $T_e(\text{ISR}_{rx})$ of the frame-based routing for CAN, FlexRay, and Ethernet is 1, 19, and 1 μ s, respectively. The $T_e(\text{ISR}_{rx})$ of the PDU-based routing and of the signal-based routing last 4 and 8 μ s more than frame-based routing (e.g., $T_e(\text{ISR}_{rx})$ of PDU-based routing for CAN is 5 μ s and $T_e(\text{ISR}_{rx})$ of the signal-based routing for CAN is 9 μ s). The worst cases for T_{cc_tx} for CAN, FlexRay, and Ethernet are

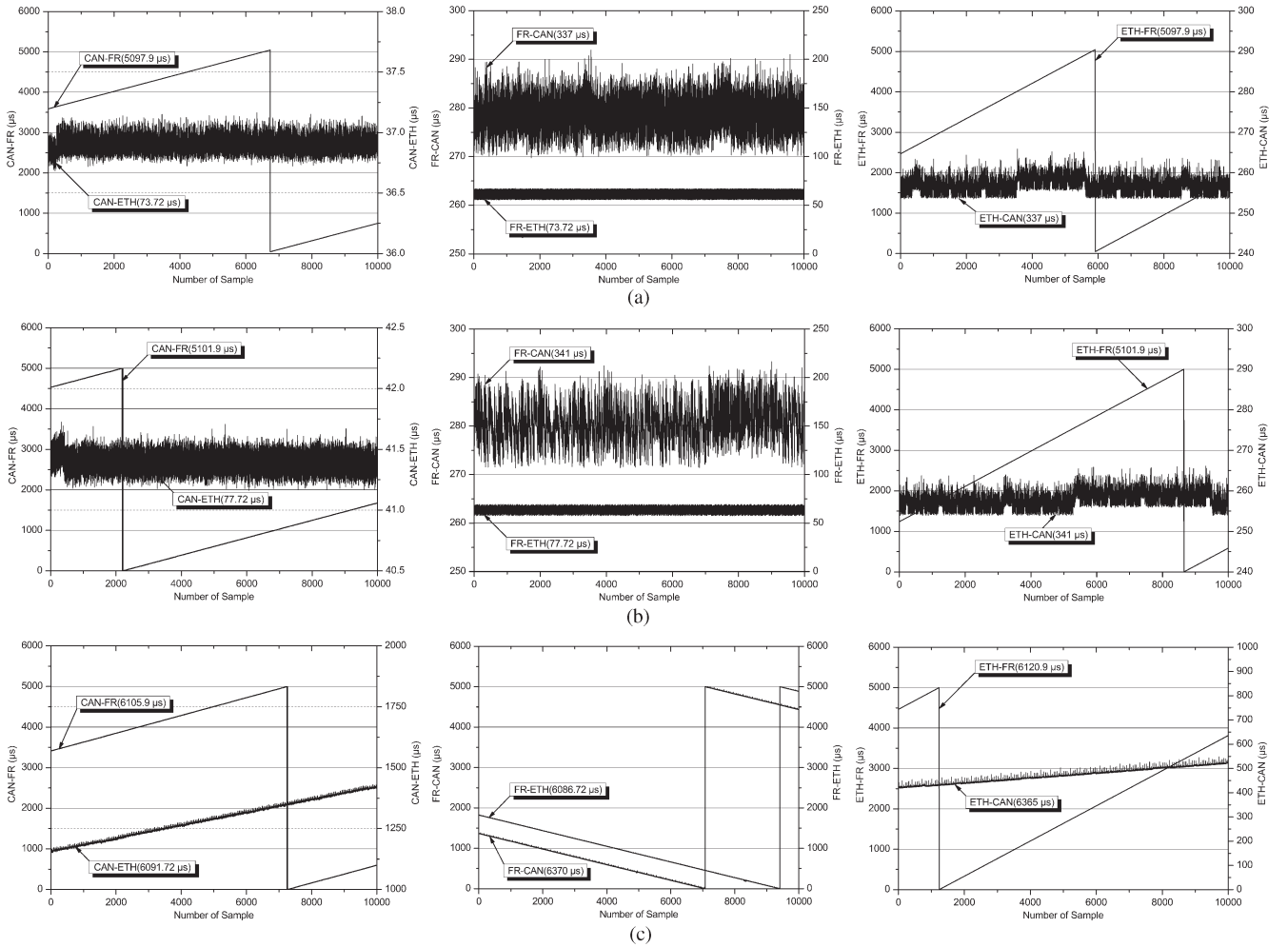


Fig. 13. Measurement results of the end-to-end processing time for frame-based routing, PDU-based routing, and signal-based routing. (a) Frame-based routing. (b) PDU-based routing. (c) Signal-based routing.

270, 30.9, and 6.72 μs , respectively. T_{protocol} for FlexRay is 5000 μs , which is the communication cycle of FlexRay, and T_{protocol} for CAN and Ethernet can be ignored (i.e., its value is zero). To simplify the analysis, we assume that only the receive interrupts for CAN, FlexRay, and Ethernet are used without supporting multiple-interrupt request from the same interrupt source, and $\text{Task}_{\text{proc}}$ is not preempted. The worst case of the end-to-end processing time occurs when the gateway receives a new message during execution of a nonpreemptive task and two interrupts, which are requested from different interrupt sources before the message is received, have been already requested. In this case, the new message can be processed after executing the nonpreemptive task and the two interrupts. For example, the worst case end-to-end processing time for the frame-based routing between CAN and Ethernet is the summation of Task_{np} (5 μs), the two interrupts (9 and 27 μs), the interrupt for the new message (1 μs), $(\theta + T_c)$ (5 μs), Task_{tx} (20 μs), and $T_{\text{cc-tx}}$ (6.72 μs), which is equal to 73.72 μs . The measured end-to-end processing time is less than the analytical results calculated for every routing method, as shown in Fig. 13.

The measured end-to-end processing time is divided into two categories, i.e., periodic and event. The periodic category represents routing methods that if the converted data is transmitted to

the destination network periodically (e.g., signal-based routing or indirect PDU-based routing that is configured in a periodic transmission mode and all FlexRay routing). CAN to FlexRay and Ethernet to FlexRay for frame-based routing [see CAN-FR and ETH-FR in Fig. 13(a)], PDU-based routing [see CAN-FR and ETH-FR in Fig. 13(b)], and all routing in signal-based routing [see Fig. 13(c)] are included in the periodic category. In this category, the end-to-end processing time is mostly affected by the transmission period of the destination message because the converted data are transmitted to the destination network according to the configured transmission period, regardless of the time at which the source message is received. Therefore, the transmission period should be shortened to reduce the end-to-end processing time for this category. Typically, the end-to-end processing time of this category seems to be less than the transmission period; however, the worst-case end-to-end processing time is longer than the transmission period, as shown in Fig. 13. The end-to-end processing time can increase or decrease because the clock drift problem occurs in a manner similar to that of the diagnostic routing between Ethernet and FlexRay. In this measurement, the end-to-end processing time increases because the clock of the senders, which are the additional embedded systems in this paper, is faster than the

gateway. The event category represents routing methods that of the converted data is transmitted to the destination network immediately (e.g., frame-based routing and direct PDU-based routing for CAN and Ethernet). Every routing method that of the destination network is CAN or Ethernet for the frame-based and PDU-based routing is included in the event category. In the case where the destination network is CAN [see FR-CAN and ETH-CAN in Fig. 13(a) and (b)], the transmission time (up to 270 μ s for 500 Kb/s CAN) is occupied by the largest part of the end-to-end processing time due to the low transmission speed of CAN, and the measured time spent by the gateway is less than 60 μ s and analytical worst case is less than 102 μ s. The end-to-end processing time can be longer for multiple times of the transmission time due to the ID-based arbitration mechanism of CAN. Therefore, the delay caused by the ID-based arbitration mechanism can be a factor that has the most significant impact on the end-to-end processing time and should be minimized. In this case, the network must be optimized to maintain a suitable bus load and prevent bursty transmission, and an increase in the priority of the destination message also prevents delays but should be carefully considered because it can delay other messages on the destination network. In the case where the destination network is Ethernet [see CAN-ETH and FR-ETH in Fig. 13(a) and (b)], the end-to-end processing time is very short because the transmission time for Ethernet (6.72 μ s) is much shorter than that for CAN (270 μ s) although the time spent by the gateway is similar to the case where the destination network is CAN.

This experiment indicates that the transmission period and the transmission delay caused by ID-based arbitration occupy a large part of the end-to-end processing time of the gateway, and these can be reduced by network optimization. Although efforts to further optimize the gateway framework are required, the time spent by the gateway framework is relatively small and can be easily reduced by replacing the processor with another that provides better computing power because the cost of the processor will decline over time according to Moore's law. Therefore, to improve the gateway performance, the network should be carefully designed and optimized by considering both the features of the gateway and the connected networks such as CAN, FlexRay, and Ethernet.

IV. CONCLUSION

This paper has proposed a gateway framework that supports CAN, FlexRay, and Ethernet. The gateway framework is independent of the hardware/software platform, and a gateway can be easily developed and verified by reusing the proposed gateway framework to reduce development costs and time. We demonstrate that the proposed gateway framework is reusable, safe, and effective by evaluating the proposed gateway framework using experimental verification. The theoretical analysis and the experimental measurements are presented to evaluate the end-to-end processing time of the gateway. From this analysis, we found that efforts to optimize the gateway framework are required; however, the efforts to optimize the network are more important to improve the performance of the gateway. The proposed gateway framework is useful for a current automotive

system that uses a central gateway and will be more useful for a future automotive system that uses DCUs as a gateway. In future work, we will investigate a security gateway that isolates the unsafe network domain (V2X) and the safe network domain (IVN) and prevent security threats endanger the safety of the vehicle.

REFERENCES

- [1] R. M. Daoud, H. H. Amer, H. M. Elsayed, and Y. Sallez, "Ethernet-based car control network," in *Proc. Can. Conf. Elect. Comput. Eng.*, May 2006, pp. 1031–1034.
- [2] N. Navet, Y. Song, F. Simonot-Lion, and C. Wilwert, "Trends in automotive communication systems," *Proc. IEEE*, vol. 93, no. 6, pp. 1204–1223, Jun. 2005.
- [3] D. Paret, *Multiplexed Networks for Embedded Systems: CAN, LIN, FlexRay, Safe-by-Wire*. Chichester, U.K.: Wiley, 2007.
- [4] G. Cena, A. Valenzano, and S. Vitturi, "Advances in automotive digital communications," *Comput. Std. Interfaces*, vol. 27, no. 6, pp. 665–678, Jun. 2005.
- [5] R. Makowitz and C. Temple, "FlexRay—A communication network for automotive control systems," in *Proc. IEEE Int. WFCs*, 2006, pp. 207–212.
- [6] G. Cena and A. Valenzano, "FastCAN: A high-performance enhanced CAN-like network," *IEEE Trans. Ind. Electron.*, vol. 47, no. 4, pp. 951–963, Aug. 2000.
- [7] G. Cena and A. Valenzano, "Achieving round-robin access in controller area networks," *IEEE Trans. Ind. Electron.*, vol. 49, no. 6, pp. 1202–1213, Dec. 2002.
- [8] A. Grzempa, *MOST: The Automotive Multimedia Network*. Poing, Germany: Franzis Verlag, 2006.
- [9] J. Nobauer, "Is Ethernet the rising star for in-vehicle networks?" in *Proc. IEEE Int. Conf. ETFA*, 2011, pp. 1–36.
- [10] A. Kern, "Ethernet and IP for automotive E/E-architectures," Ph.D. dissertation, Dept. Comput. Sci., Univ. Erlangen-Nuremberg, Erlangen, Germany, 2012.
- [11] H. T. Lim, L. Volker, and D. Herrscher, "Challenges in a future IP/Ethernet-based in-car network for real-time applications," in *Proc. 48th DAC*, 2011, pp. 7–12.
- [12] L. L. Bello, "The case for Ethernet in automotive communications," *ACM SIGBED Rev.*, vol. 8, no. 4, pp. 7–15, Dec. 2011.
- [13] B. Triess and T. Hogenmueller, "Ethernet gateway for automotive," in *Proc. 2nd Ethernet IP@ Autom. Technol. Day*, 2012, pp. 1–26.
- [14] S. Singer, Next Generation Automotive Gateways, Jul. 14, 2009. [Online]. Available: http://www.freescale.com/files/training_pdf/VFTF09_AA130.pdf
- [15] J. Sommer and R. Blind, "Optimized resource dimensioning in an embedded CAN-CAN gateway," in *Proc. IEEE Int. SIES*, 2007, pp. 55–62.
- [16] S.-H. Seo, J. H. Kim, S.-H. Hwang, K. H. Kwon, and J. W. Jeon, "A reliable gateway for in-vehicle networks based on LIN, CAN, and FlexRay," *ACM Trans. Embedded Comput. Syst.*, vol. 11, no. 1, p. 7, Mar. 2012.
- [17] T. Herpel, B. Kloiber, R. German, and S. Fey, "Routing of safety-relevant messages in automotive ECU networks," in *Proc. IEEE VTC—Fall*, 2009, pp. 1–5.
- [18] O. Sander, M. Hubner, J. Becker, and M. Traub, "Reducing latency times by accelerated routing mechanisms for an FPGA gateway in the automotive domain," in *Proc. Int. Conf. FPT*, 2008, pp. 97–104.
- [19] S.-H. Seo *et al.*, "A fault-tolerant gateway for in-vehicle networks," in *Proc. IEEE Int. Conf. INDIN*, 2008, pp. 1144–1148.
- [20] H. Zinner, J. Noebauer, T. Gallner, J. Seitz, and T. Waas, "Application and realization of gateways between conventional automotive and IP/Ethernet-based networks," in *Proc. 48th DAC*, 2011, pp. 1–6.
- [21] S. Shaheen, D. Heffernan, and G. Leen, "A gateway for time-triggered control networks," *Microprocess. Microsyst.*, vol. 31, no. 1, pp. 38–50, Feb. 2007.
- [22] AUTOSAR—Automotive Open System Architecture Rel. 4.1. [Online]. Available: <http://www.autosar.org>
- [23] *Road Vehicles—Open Interface for Embedded Automotive Applications*, ISO Std. 17356, 2005.
- [24] J. Lemieux, *Programming in the OSEK/VDX Environment*. San Francisco, CA, USA: CMP, 2001.
- [25] *Road Vehicles—Diagnostic Communication Over Internet Protocol (DoIP)*, ISO Std. 13400, 2012.
- [26] *Road Vehicles—Unified Diagnostic Service (UDS)*, ISO Std. 14229, 2013.

- [27] AUTOSAR—Specification of CAN Network Management V3.5.0 R4.1 Rev 2. [Online]. Available: <http://www.autosar.org>
- [28] AUTOSAR—Layered Software Architecture V3.4.0 Rev 3. [Online]. Available: <http://www.autosar.org>
- [29] AUTOSAR—Specification of Communication V5.1.0 R4.1 Rev 2. [Online]. Available: <http://www.autosar.org>
- [30] AUTOSAR—Specification of CAN Transport Layer V5.0.0 R4.1 Rev 1. [Online]. Available: <http://www.autosar.org>
- [31] AUTOSAR—Specification of FlexRay AUTOSAR Transport Layer V3.1.0 R4.1 Rev 1. [Online]. Available: <http://www.autosar.org>
- [32] *Road Vehicles—Unified Diagnostic Service (UDS)—Part 3: Unified Diagnostic Services on CAN Implementation (UDSonCAN)*, ISO Std. 14229-3, 2013.
- [33] *Road vehicles—Unified Diagnostic Service (UDS)—Part 4: Unified Diagnostic Services on FlexRay Implementation (UDSonFR)*, ISO Std. 14229-4, 2012.
- [34] *ASAM MCD-2 NET (Market Name: FIBEX) V4.1.0*, ASAM Std., 2013.
- [35] MPC5668GKIT : MPC5668G/E Evaluation Kit. [Online]. Available: http://www.freescale.com/webapp/sps/site/prod_summary.jsp?code=MPC5668GKIT
- [36] R. Davis and N. Navet, Controller Area Network (CAN) Schedulability Analysis for Messages with Arbitrary Deadlines in FIFO and Work-Conserving Queues, Jun. 29, 2012. [Online]. Available: http://nicolas.navet.eu/seminars/CAN_WQ_Presentation_Lux.pdf
- [37] D. Engler and M. Madanlal, "Static analysis versus software model checking for bug finding," in *Proc. Int. Conf. VMCAI*, 2004, pp. 191–210.
- [38] N. Ayewah, D. Hovemeyer, J. D. Morgenthaler, and J. Penix, "Using static analysis to find bugs," *IEEE Softw.*, vol. 25, no. 5, pp. 22–29, Sep. 2008.
- [39] L. Hatton, "Language subsetting in an industrial context: A comparison of MISRA C 1998 and MISRA C 2004," *Inform. Softw. Technol.*, vol. 49, no. 5, pp. 475–482, May 2007.
- [40] L. Hatton, "Safer language subsets: An overview and a case history, MISRA C," *Inf. Softw. Technol.*, vol. 46, no. 7, pp. 465–472, Jun. 2004.
- [41] Guidelines for the Use of the C Language in Critical Systems, MISRA, Warwickshire, U.K., Oct. 2004. [Online]. Available: <http://misra.org.uk>
- [42] CodeSonar-Static Analysis Tool. [Online]. Available: <http://www.grammtech.com/codesonar>
- [43] R. I. Davis, A. Burns, R. J. Bril, and J. J. Lukkien, "Controller area network (CAN) schedulability analysis: Refuted, revisited, and revised," *Real-Time Syst.*, vol. 35, no. 3, pp. 239–272, Apr. 2007.
- [44] K. Schmidt and E. G. Schmidt, "Message scheduling for the FlexRay protocol: The static segment," *IEEE Trans. Veh. Technol.*, vol. 58, no. 5, pp. 2170–2179, Jun. 2009.
- [45] E. G. Schmidt and K. Schmidt, "Message scheduling for the FlexRay protocol: The dynamic segment," *IEEE Trans. Veh. Technol.*, vol. 58, no. 5, pp. 2160–2169, Jun. 2009.
- [46] M. Lukasiewicz, M. Glab, J. Teich, and P. Milbredt, "FlexRay schedule optimization of the static segment," in *Proc. 7th IEEE/ACM Int. Conf. Hardware/Softw. Codesign Syst. Synthesis*, 2009, pp. 363–372.
- [47] H. Zeng, M. D. Natale, A. Ghosal, and A. S-Vincentelli, "Schedule optimization of time-triggered systems communicating over the FlexRay static segment," *IEEE Trans. Ind. Informat.*, vol. 7, no. 1, pp. 1–17, Feb. 2011.
- [48] T. Pop, P. Pop, P. Else, Z. Peng, and A. Andrei, "Time analysis of the FlexRay communication protocol," *Real-Time Syst.*, vol. 28, no. 1–3, pp. 205–235, Oct. 2007.
- [49] A. Hagiescu *et al.*, "Performance analysis of FlexRay-based ECU networks," in *Proc. 44th DAC*, 2007, pp. 284–289.
- [50] F. Sivrikaya and B. Yener, "Time synchronization in sensor networks: A survey," *IEEE Netw.*, vol. 18, no. 4, pp. 45–50, Jul. 2004.



Jin Ho Kim received the B.S. and M.S. degrees in 2007 and 2009, respectively, from Sungkyunkwan University, Suwon, Korea, where he is currently working toward the Ph.D. degree, all in electronic, electrical, and computer engineering.

His research interests include in-vehicle networks, real-time Ethernet, automotive open system architecture (AUTOSAR), and real-time embedded systems.



Suk-Hyun Seo received the B.S., M.S., and Ph.D. degrees from Sungkyunkwan University, Suwon, Korea, in 2005, 2007, and 2011, respectively, all in electronic, electrical, and computer engineering.

He is currently a Senior Researcher with Hyundai Motors, Hwaseong, Korea. His research interests include in-vehicle networks, electrical/electronic architecture, electronic control unit, and automotive open system architecture (AUTOSAR).



Nguyen-Tien Hai received the Engineer diploma in software engineering from Tula State University, Tula, Russia, in 2011. He is currently pursuing the Ph.D. degree in electronic, electrical, and computer engineering with the Automation Lab, Sungkyunkwan University, Suwon, Korea.

His research interests include in-vehicle networks and real-time embedded systems.



Bo Mu Cheon received the B.S. degree in information and communication engineering from Kongju National University, Cheonan, Korea, in 2013. He is currently working toward the M.S. degree in electronic, electrical, and computer engineering with Sungkyunkwan University, Suwon, Korea.

His research interests include embedded systems for automotive and in-vehicle networks.



Young Seo Lee received the B.S. degree in information and communication engineering from Kongju National University, Cheonan, Korea, in 2014. He is currently working toward the M.S. degree in electronic, electrical, and computer engineering with Sungkyunkwan University, Suwon, Korea.

His research interests include embedded systems for automotive and in-vehicle networks.



Jae Wook Jeon (S'82–M'84) received the B.S. and M.S. degrees in electronics engineering from Seoul National University, Seoul, Korea, in 1984 and 1986, respectively, and the Ph.D. degree in electrical engineering from Purdue University, West Lafayette, IN, USA, in 1990.

From 1990 to 1994, he was a Senior Researcher with Samsung Electronics, Suwon, Korea. Since 1994, he has been with Sungkyunkwan University, Suwon, where he was first an Assistant Professor with the School of Electrical and Computer Engineering and is currently a Professor with the School of Information and Communication Engineering. His research interests include robotics, embedded systems, and factory automation.